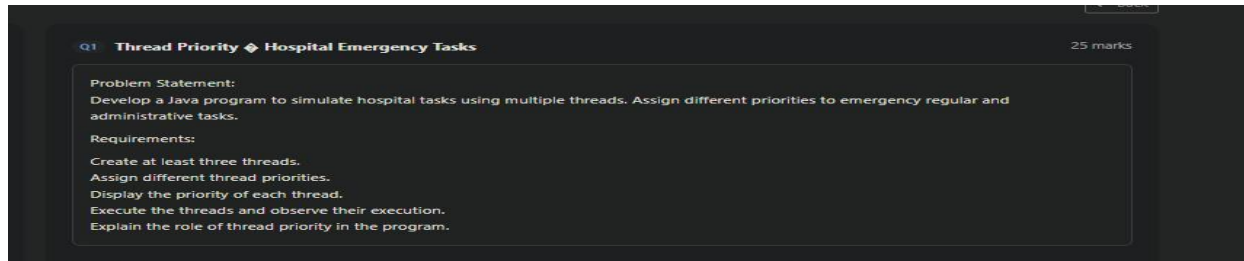


CO3_AT3

Java Programs CSA0911

AL NAJEEB A | 192424325 | SLOT B

Q1 Thread Priority *Hospital Emergency Tasks*



The screenshot shows a problem statement for a Java programming task. The title is 'Q1 Thread Priority Hospital Emergency Tasks' with a '25 marks' indicator. The problem statement asks to develop a Java program to simulate hospital tasks using multiple threads, assigning different priorities to emergency, regular, and administrative tasks. The requirements are: create at least three threads, assign different thread priorities, display the priority of each thread, execute the threads and observe their execution, and explain the role of thread priority in the program.

Q1 Thread Priority Hospital Emergency Tasks 25 marks

Problem Statement:
Develop a Java program to simulate hospital tasks using multiple threads. Assign different priorities to emergency regular and administrative tasks.

Requirements:
Create at least three threads.
Assign different thread priorities.
Display the priority of each thread.
Execute the threads and observe their execution.
Explain the role of thread priority in the program.

```
public class Main{
    public static void main(String[] args){

        Thread emergency = new Thread(new Runnable(){
            public void run(){
                System.out.println("Emergency Task running - Priority: " +
Thread.currentThread().getPriority());
            }
        });

        Thread regular = new Thread(new Runnable(){
            public void run(){
                System.out.println("Regular Task running - Priority: " +
Thread.currentThread().getPriority());
            }
        });

        Thread admin = new Thread(new Runnable(){
            public void run(){
                System.out.println("Admin Task running - Priority: " +
Thread.currentThread().getPriority());
            }
        });

        emergency.setPriority(Thread.MAX_PRIORITY);
        regular.setPriority(Thread.NORM_PRIORITY);
        admin.setPriority(Thread.MIN_PRIORITY);

        emergency.start();
        regular.start();
        admin.start();
    }
}
```

OUTPUT

Output

```
Emergency Task running - Priority: 10
Regular Task running - Priority: 5
Admin Task running - Priority: 1
```

Q2 Multithreaded Number Processing

← Back

Q2 Multithreaded Number Processing

25 marks

Problem Statement:
Develop a Java program that creates multiple threads to perform different operations on a set of numbers.

Requirements:
Create separate threads to calculate the sum average and maximum value.
Use the Thread class.
Execute the threads concurrently.
Display the result produced by each thread

```
public class Main{
    static int[] nums = {10, 20, 30, 40, 50};

    public static void main(String[] args){

        Thread sumThread = new Thread(new Runnable(){
            public void run(){
                int sum = 0;
                for(int n : nums) sum = sum + n;
                System.out.println("Sum: " + sum);
            }
        });

        Thread avgThread = new Thread(new Runnable(){
            public void run(){
                int sum = 0;
                for(int n : nums) sum = sum + n;
                System.out.println("Average: " + (sum / nums.length));
            }
        });

        Thread maxThread = new Thread(new Runnable(){
            public void run(){
                int max = nums[0];
                for(int n : nums) if(n > max) max = n;
                System.out.println("Max: " + max);
            }
        });

        sumThread.start();
        avgThread.start();
        maxThread.start();
    }
}
```

OUTPUT

Output

```
Sum: 150
Average: 30
Max: 50
```

Q3 Student Grade Calculator *ArithmeticException*

Q3 Student Grade Calculator *ArithmeticException*

25 marks

Problem Statement:
Develop a Java program to calculate the average marks of a student. The program should accept marks obtained and the number of subjects from the user. If the number of subjects entered is 0 handle the exception gracefully.

Requirements:
Read total marks and number of subjects.
Calculate and display the average.
Handle `ArithmeticException` using try-catch.
Display a user-friendly error message for division by zero.

```
public class Main{
    public static void main(String[] args){

        int totalMarks = 450;
        int subjects = 0;

        try{
            if(subjects == 0){
                throw new ArithmeticException("Division by zero");
            }
            double avg = totalMarks / subjects;
            System.out.println("Average Marks: " + avg);
        }
        catch(ArithmeticException e){
            System.out.println("Error: Number of subjects cannot be zero.");
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

OUTPUT

Output

```
Error: Number of subjects cannot be zero.
Exception: Division by zero
```

Q4 Food Ordering System *Multiple Threads*

Q4 Food Ordering System ↗ Multiple Threads

25 marks

Problem Statement:
Develop a multithreaded Java program for a food ordering system. Separate threads should process order placement food preparation and delivery.

Requirements:
Create three separate threads.
Each thread should perform its respective task.
Display the execution status of each task.
Demonstrate concurrent execution.

Your Code

Input

```
public class Main{
    public static void main(String[] args){

        Thread orderPlacement = new Thread(new Runnable(){
            public void run(){
                System.out.println("Order Placement: Order received.");
            }
        });

        Thread foodPreparation = new Thread(new Runnable(){
            public void run(){
                System.out.println("Food Preparation: Food is being prepared.");
            }
        });

        Thread delivery = new Thread(new Runnable(){
            public void run(){
                System.out.println("Delivery: Food is on the way.");
            }
        });

        orderPlacement.start();
        foodPreparation.start();
        delivery.start();
    }
}
```

OUTPUT

Output

```
Order Placement: Order received.
Food Preparation: Food is being prepared.
Delivery: Food is on the way.
```